RAPPORT INTERMÉDIAIRE PSC X24

Architecture Agentique et Hybride pour l'Extraction de Connaissances sur Code Propriétaire (Envision)

18 janvier 2026

Dégot–Silvestre Gaétan
Dorchies Yoan
Kamdem Ivann
Thébault Guilhem
Guediche Adam

TABLE DES MATIÈRES

1 Introduction : Le Défi des DSLs Industriels **1**
1.1 Contexte : L'angle mort des Grands Modèles de Langage . . . 1
1.2 L'intuition du projet : RAG plutôt que Fine-Tuning . . . 1
1.3 Objectifs du rapport intermédiaire . . . 1

2 État de l'art : De la prédiction de mots à l'Agentification **2**
2.1 Les limites des LLM "figés" et l'Hallucination . . . 2
2.2 Le paradigme RAG (Retrieval-Augmented Generation) . . . 2
2.3 L'émergence des Agents Autonomes (Agentic AI) . . . 2

3 Méthodologie et Architecture Implémentée **4**
3.1 Préparation des données : Du code brut aux "Chunks" sémantiques . . . 4
3.1.1 Le problème du découpage standard . . . 4
3.1.2 Notre solution : Le Semantic Chunker . . . 4
3.2 Stratégie de Recherche Hybride : Le Routeur . . . 4
3.2.1 La limite du Vectoriel face au Lexical . . . 4
3.2.2 Implémentation du Routeur Intelligent . . . 5
3.3 Architecture Agentique sous LangGraph . . . 5
3.3.1 Le flux de travail (Workflow) . . . 5
3.3.2 Fonctionnement de la boucle . . . 5

4 Protocole d'Évaluation (Benchmarking) **6**
4.1 Le défi de la métrique : L'illusion de la similarité . . . 6
4.2 La solution méthodologique : "LLM-as-a-Judge" . . . 6
4.2.1 Constitution du Benchmark (Golden Dataset) . . . 6
4.3 Métriques de performance système (Monitoring) . . . 7

5 Gestion de Projet et Perspectives **8**
5.1 Bilan d'étape : Comparaison Prévu vs Réalisé . . . 8
5.1.1 L'évolution de la Pipeline . . . 8
5.2 Défis techniques rencontrés et solutions implémentées . . . 8
5.2.1 La gestion de la fenêtre de contexte (Context Window) . . . 8
5.2.2 La résolution des chemins dynamiques (Dynamic Paths) . . . 8
5.2.3 La gestion des identifiants fichiers (Mapping) . . . 9
5.2.4 L'instabilité du raisonnement agentique . . . 9
5.3 Vers une industrialisation : Métriques et Modèles . . . 9
5.3.1 Définition de métriques de performance . . . 9
5.3.2 Choix des modèles et contraintes de coûts . . . 9
5.4 Pistes d'amélioration avancées pour le RAG . . . 10
5.5 Feuille de route pour la finalisation . . . 10

1
INTRODUCTION : LE DÉFI DES DSLS INDUSTRIELS

1.1 Contexte : L'angle mort des Grands Modèles de Langage

L'avènement des modèles de langage massivement pré-entraînés (LLM) a bouleversé les pratiques de développement logiciel. Sur des langages populaires comme Python ou Javascript, des outils comme GitHub Copilot agissent désormais comme des assistants quasi-omniscients. La raison de ce succès est simple : ces langages constituent une part significative du corpus d'entraînement public sur lequel ces modèles ont appris.

Cependant, la réalité industrielle est plus nuancée. De nombreuses entreprises, à l'instar de notre partenaire **Lokad**, fondent leur avantage compétitif sur des *Domain Specific Languages* (DSL) propriétaires. Dans le cas de Lokad, il s'agit d'**Envision** [1], un langage dédié à l'optimisation de la Supply Chain, combinant des primitives de manipulation de vecteurs et d'algèbre relationnelle.

Ici réside le cœur de notre problématique : les LLM standards (GPT-4, Mistral, Llama) n'ont quasiment jamais "lu" de code Envision. Lorsqu'ils sont interrogés sur ce langage, ils souffrent d'un phénomène d'**hallucination structurelle** : ils inventent une syntaxe plausible — ressemblant souvent à un mélange de SQL et de Python — mais fonctionnellement incorrecte.

1.2 L'intuition du projet : RAG plutôt que Fine-Tuning

Face à ce déficit de connaissance, deux approches s'opposent classiquement :

1. **Le Fine-Tuning (Réapprentissage) :** On réentraîne le modèle sur le corpus privé. Cette méthode est coûteuse, lente, et rigide (le modèle est obsolète dès que la documentation change).
2. **Le RAG (Retrieval-Augmented Generation) :** On ne touche pas au cerveau du modèle, mais on lui fournit, au moment de la requête, les pages du manuel pertinentes.

Ce PSC prend le parti de la seconde approche, jugée plus agile pour un contexte industriel. Cependant, notre travail préliminaire a montré qu'un RAG "naïf" (simplement chercher des documents par similarité) ne suffit pas pour du code. Une question du type *« Quels scripts modifient la variable SupplyLeadTime ? »* nécessite une précision absolue qu'une recherche sémantique floue ne peut garantir.

1.3 Objectifs du rapport intermédiaire

L'objectif de ce projet a donc évolué : il ne s'agit plus seulement de créer un moteur de recherche, mais de concevoir une **architecture agentique hybride**. Ce système doit être capable de décider de lui-même s'il doit effectuer une recherche sémantique (comprendre un concept) ou lexicale (trouver une variable exacte), et d'itérer sur sa propre réponse.

Ce rapport intermédiaire présente l'état actuel de ce système. Nous détaillerons comment nous sommes passés d'une pipeline linéaire à l'utilisation de graphes d'états (via *LangGraph*), comment nous avons surmonté les limites du découpage de code (*chunking*), et comment nous avons mis en place un protocole d'évaluation automatisé (*LLM-as-a-Judge*) pour mesurer nos progrès de manière objective.

2
ÉTAT DE L'ART : DE LA PRÉDICTION DE MOTS À L'AGENTIFICATION

Pour comprendre les choix architecturaux présentés dans ce rapport, il est nécessaire de retracer l'évolution rapide des techniques d'interaction avec les LLM, en se focalisant sur leur application au code informatique.

2.1 Les limites des LLM "figés" et l'Hallucination

Depuis l'introduction de l'architecture *Transformer* [4], la capacité des modèles à générer du texte cohérent a explosé. Cependant, ces modèles sont statiques : leurs connaissances s'arrêtent à la date de leur fin d'entraînement (*knowledge cutoff*).

Face à un langage propriétaire comme Envision, le modèle se trouve en situation de *Zero-Shot* (aucun exemple) sur une tâche inconnue. Il applique alors des schémas probabilistes issus d'autres langages. Comme le soulignent Tauman et al. [6], ces hallucinations sont d'autant plus pernicieuses qu'elles sont formulées avec une grande certitude apparente. Le *Prompt Engineering* classique (optimiser la formulation de la question) [8] atteint ici ses limites : on ne peut pas demander au modèle de "mieux réfléchir" sur un sujet qu'il ignore totalement.

2.2 Le paradigme RAG (Retrieval-Augmented Generation)

Introduit formellement par Lewis et al. [5], le RAG est devenu le standard pour pallier le manque de connaissances spécifiques. Le principe est de coupler le générateur (LLM) à un récupérateur (*Retriever*).

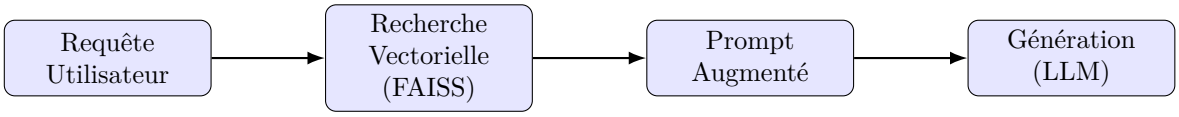

Figure 1 – Architecture RAG linéaire classique

Généralement, le RAG repose sur des **embeddings** : le texte est transformé en vecteurs mathématiques. Si deux vecteurs sont proches dans l'espace, les textes sont supposés avoir un sens proche [7]. Cependant, notre expérience montre une limite critique de cette approche pour le code : la recherche vectorielle capture le "sens" mais perd la "précision". Elle confondra facilement `Stock.Available` et `Stock.OnOrder`, car sémantiquement proches, alors qu'en programmation, la distinction doit être exacte.

2.3 L'émergence des Agents Autonomes (Agentic AI)

C'est pour répondre à ces limitations qu'émerge aujourd'hui l'**IA Agentique**. Contrairement à un RAG linéaire (une seule passe), un Agent est un système qui possède :

- **Des Outils (Tools) :** Il ne se contente pas de prédire du texte, il peut exécuter des fonctions (lancer une recherche Google, faire un calcul, exécuter une commande GREP).
- **Une Planification (Reasoning) :** Il peut décomposer une tâche complexe en sous-tâches.
- **Une Mémoire et une Réflexion :** Il peut critiquer son propre résultat et recommencer si nécessaire.

Des frameworks récents comme **LangGraph** [2] permettent de coder ces comportements sous forme de graphes cycliques. Plutôt que de forcer le modèle à répondre immédiatement, on lui permet d'entrer dans une boucle de pensée (*Chain of Thought*) où il peut affiner sa recherche d'information avant de formuler une réponse. C'est cette approche, inspirée notamment de travaux sur la génération de code DSL [3], qui constitue le cœur technique de notre projet actuel.

3
MÉTHODOLOGIE ET ARCHITECTURE IMPLÉMENTÉE

Notre approche ne s'est pas construite en une seule étape. Elle est le fruit d'itérations successives, partant d'un constat d'échec sur les méthodes classiques pour aboutir à une architecture hybride et agentique. Cette section détaille les trois piliers techniques de notre solution actuelle.

3.1 Préparation des données : Du code brut aux "Chunks" sémantiques

La qualité d'un système RAG dépend avant tout de la qualité de ses données. Contrairement à du texte littéraire, le code source d'Envision possède une structure hiérarchique forte qui ne supporte pas un découpage arbitraire.

3.1.1 • Le problème du découpage standard

Les méthodes classiques de *chunking* (découpage par nombre fixe de tokens ou par séparateurs de paragraphes) s'avèrent destructrices pour du code.

- Si un découpage coupe une fonction en deux, le modèle perd la signature de la fonction (ses arguments) ou son corps logique.
- Le contexte est souvent situé en début de fichier (déclaration des tables), information cruciale pour comprendre les lignes situées 500 lignes plus bas.

3.1.2 • Notre solution : Le Semantic Chunker

Nous avons développé un module de parsing spécifique (`EnvisionParser`) qui analyse la structure syntaxique des fichiers `.nvn`. Plutôt que de découper aveuglément, nous identifions des blocs logiques :

- **Blocs de définition :** `table`, `expect`, `read`.
- **Blocs logiques :** Délimitations par les commentaires spéciaux `///` utilisés dans Envision.

Le `SemanticChunker` regroupe ensuite ces blocs en unités cohérentes, en s'assurant que chaque chunk conserve suffisamment de métadonnées (nom du fichier source, imports) pour être compréhensible de manière isolée par le LLM.

3.2 Stratégie de Recherche Hybride : Le Routeur

L'une des contributions majeures de notre architecture est de ne pas se reposer uniquement sur la recherche vectorielle. Nous avons identifié deux typologies de questions radicalement différentes, nécessitant des outils distincts.

3.2.1 • La limite du Vectoriel face au Lexical

La recherche vectorielle (via FAISS et des embeddings type *all-MiniLM-L6-v2*) excelle pour comprendre des concepts (*« Comment calculer le stock ? »*). Cependant, elle échoue souvent sur des requêtes précises (*« Où est utilisée la variable 'MySpecificVar' ? »*). Deux noms de variables différents peuvent avoir des vecteurs très proches s'ils apparaissent dans des contextes similaires, menant à des hallucinations.

3.2.2 • Implémentation du Routeur Intelligent

Nous avons introduit un composant **Router** en amont de la récupération. Il s'agit d'un appel léger au LLM qui classifie l'intention de l'utilisateur :

1. **Intent "Conceptuel" :** Activation du **Retriever FAISS** (Recherche sémantique).
2. **Intent "Exact" :** Activation du **Retriever GREP**. Ce module exécute des expressions régulières (Regex) directement sur le code source pour une précision de 100%.

Cette hybridité permet au système de bénéficier du "meilleur des deux mondes" : la compréhension du sens et la précision chirurgicale de la recherche textuelle.

3.3 Architecture Agentique sous LangGraph

Plutôt qu'une pipeline linéaire (Question → Recherche → Réponse), nous avons implémenté une architecture cyclique basée sur des graphes d'états (*StateGraph*), utilisant la librairie **LangGraph** [2].

3.3.1 • Le flux de travail (Workflow)

Le système est modélisé comme un agent capable de raisonner et d'utiliser des outils.

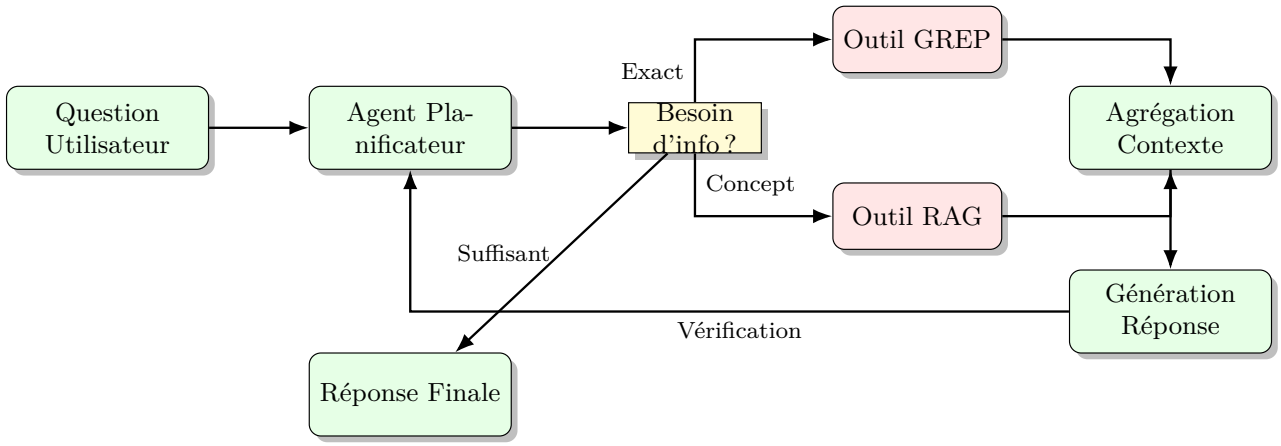

Figure 2 – Architecture du flux agentique implémenté avec LangGraph

3.3.2 • Fonctionnement de la boucle

Le cycle se déroule comme suit :

1. **Planification :** L'agent analyse la demande. S'il lui manque des infos, il décide d'appeler un outil (Tool Call).
2. **Exécution :** L'outil (RAG ou GREP) s'exécute et retourne des données brutes.
3. **Observation et Réflexion :** L'agent reçoit ces données. Il peut décider qu'elles sont suffisantes et générer la réponse, ou qu'elles sont incomplètes et formuler une nouvelle requête (ex : "J'ai trouvé la table, maintenant je cherche la colonne").

Cette capacité de *Backtracking* (retour en arrière) et d'itération est ce qui distingue fondamentalement notre prototype d'un simple moteur de recherche : il mime le comportement d'un développeur humain qui navigue dans le code.

4
PROTOCOLE D'ÉVALUATION (BENCHMARKING)

Le développement d'une architecture agentique complexe ne constitue que la moitié du travail d'ingénierie. L'autre moitié, tout aussi cruciale, réside dans la capacité à mesurer objectivement la performance du système. L'évaluation automatique de systèmes de génération de code (*Code Generation*) est un domaine notoirement difficile, car contrairement à des tâches de classification, il n'existe pas toujours une unique "bonne réponse" syntaxique pour un problème donné.

4.1 Le défi de la métrique : L'illusion de la similarité

Dans les systèmes de NLP classiques, l'étalon-or est souvent la *Cosine Similarity* (similarité cosinus). Cette métrique géométrique calcule l'angle entre le vecteur de la réponse générée et celui de la réponse de référence (Vérité Terrain). Si l'angle est faible, les textes sont considérés comme sémantiquement proches.

Cependant, dans le contexte rigoureux d'un DSL comme Envision, cette approche s'est révélée dangereuse, voire contre-productive :

- **Faux Positifs (L'illusion de justesse) :** Un modèle peut générer une réponse syntaxiquement parfaite, utilisant le bon vocabulaire technique, ce qui lui confère un score de similarité très élevé. Pourtant, une simple inversion d'opérateur (un `+` au lieu d'un `-`) ou une référence à la mauvaise table (`Orders` au lieu de `OrderLines`) rend le code fonctionnellement faux. La métrique vectorielle ne capture pas cette "rupture" logique.
- **Faux Négatifs (La pénalité injuste) :** Inversement, le modèle peut proposer une solution algorithmique différente de la référence (par exemple, utiliser une boucle explicite au lieu d'une agrégation vectorielle), mais tout à fait valide et fonctionnelle. Une comparaison purement textuelle ou vectorielle pénaliserait injustement cette créativité valide.

4.2 La solution méthodologique : "LLM-as-a-Judge"

Pour pallier ces biais structurels, nous avons implémenté une méthode d'évaluation moderne basée sur un **Juge Artificiel**, inspirée des travaux récents sur l'évaluation des chatbots [9].

Le principe est de déléguer l'évaluation à un modèle tiers (dans notre cas, un modèle performant comme **Mistral** ou un modèle spécialisé) dont l'unique tâche est de comparer la réponse de l'agent avec la vérité terrain. Le Juge ne génère pas de code ; il analyse la cohérence logique et factuelle.

4.2.1 • Constitution du Benchmark (Golden Dataset)

Pour nourrir ce juge, nous avons construit manuellement un jeu de données de test (*Golden Dataset*) composé de paires questions-réponses issues de l'historique réel de développement de Lokad. Ce dataset est segmenté en deux catégories distinctes pour tester les différentes capacités de notre architecture hybride :

1. **Questions "Structural" (Test du GREP) :**
   Exemple : "Quels scripts lisent le fichier `/Clean/Items.ion` ?"
   Attendu : Une liste exhaustive et exacte des noms de fichiers. Ici, l'évaluation est binaire : la liste est complète ou elle ne l'est pas. C'est un test de la capacité de l'agent à naviguer dans la structure du projet.
2. **Questions "Retrieval" (Test du RAG) :**
   Exemple : "Où est définie la logique d'actualisation du stock ?"

Attendu : Une réponse sémantique pointant vers les bons modules et expliquant brièvement la logique. Ici, le Juge évalue la pertinence des documents retrouvés et la clarté de la synthèse.

4.3 Métriques de performance système (Monitoring)

Au-delà de la qualité intrinsèque des réponses (précision), la viabilité industrielle de l'outil dépend de sa performance opérationnelle. Nous avons donc instrumenté notre pipeline pour suivre deux indicateurs critiques :

— **La Latence d'Inférence :** Nous nous interessons au temps écoulé entre la requête utilisateur et la réponse finale. Actuellement, une boucle agentique complète (réflexion, appel d'outil, génération) prend entre 5 et 10 secondes. C'est acceptable pour un assistant de développement, mais nous cherchons à réduire ce délai en optimisant les appels réseaux.

— **Le Temps de Construction de l'Index (Indexing Time) :** Une métrique souvent négligée mais vitale. Le code source d'Envision évolue tous les jours. Si notre système met 30 minutes à "lire" et indexer la base de code, il sera toujours obsolète.
Grâce à notre parser optimisé, nous parvenons actuellement à parser, chunker et indexer l'ensemble du corpus (plusieurs centaines de scripts) en moins de **1 minute**.

5
GESTION DE PROJET ET PERSPECTIVES

Ce rapport intermédiaire marque une étape charnière dans le déroulement de notre Projet Scientifique Collectif. Il symbolise le passage d'une phase purement exploratoire, caractérisée par la découverte du langage et des outils, à une phase de consolidation technique et d'architecture. Le projet a connu plusieurs pivots majeurs par rapport à la proposition initiale, dictés par la réalité technique complexe du code Envision et les limitations inhérentes aux modèles de langage actuels.

5.1 Bilan d'étape : Comparaison Prévu vs Réalisé

5.1.1 • L'évolution de la Pipeline

Initialement, notre vision du système reposait sur une approche classique et linéaire, telle qu'elle est souvent présentée dans la littérature académique sur le RAG. Nous avions imaginé une séquence simple :

1. **Parsing :** Lecture des fichiers.
2. **Indexation :** Création des vecteurs.
3. **Génération :** Réponse du LLM.

Cependant, la confrontation avec les cas d'usage réels a rapidement invalidé cette approche simpliste. La réalité du terrain nous a poussés vers une architecture **Agentique non-linéaire**. L'ajout du *Router* (pour discriminer les types de questions) et des boucles de rétroaction (*Feedback Loops*) via **LangGraph** ne faisait pas partie du plan initial. Ces composants se sont imposés comme des nécessités absolues pour traiter les questions complexes où une seule passe de recherche ne suffit pas à rassembler le contexte nécessaire.

5.2 Défis techniques rencontrés et solutions implémentées

Le développement de cette architecture a mis en lumière plusieurs verrous techniques spécifiques au DSL Envision, nécessitant des solutions ad-hoc parfois ingénieuses.

5.2.1 • La gestion de la fenêtre de contexte (Context Window)

Bien que les modèles récents acceptent des contextes de plus en plus larges (jusqu'à 128k tokens), injecter l'intégralité du code source d'un projet dans le prompt est une stratégie inefficace. Non seulement cela augmente drastiquement la latence et les coûts, mais cela dilue l'information pertinente dans du bruit ("Lost in the Middle phenomenon").
Solution : Nous avons développé un *Semantic Chunker*. Contrairement à un découpage brut, cet outil analyse la structure du code pour ne sélectionner et n'injecter que les fragments (fonctions, définitions de tables) strictement nécessaires à la résolution de la tâche, optimisant ainsi le rapport signal/bruit.

5.2.2 • La résolution des chemins dynamiques (Dynamic Paths)

Une difficulté majeure, apparue tardivement dans le développement, concerne la définition itérative des noms de fichiers dans Envision. Le code utilise fréquemment des interpolations de chaînes de caractères pour définir les chemins d'importation, par exemple :

```
const testStorage = "/Data/"
const inputPath = "\{testStorage}Clean/"
read "\{inputPath}Items.ion" as Items
```

Une recherche textuelle simple (GREP) sur la chaîne `"/Clean/Items.ion"` échoue ici, car le chemin complet n'est jamais écrit explicitement en une seule ligne.
Solution (Patch actuel) : Nous avons dû ajuster l'outil GREP en urgence pour gérer cette complexité. L'outil effectue désormais une résolution récursive à la volée : il isole la dernière partie du chemin, vérifie si le préfixe correspond à une variable définie précédemment (comme `{inputPath}`), et remonte ainsi la chaîne de dépendances jusqu'à la racine. Bien que fonctionnelle, cette méthode est coûteuse en temps de calcul car elle s'exécute au moment de la requête (*runtime*).

5.2.3 • La gestion des identifiants fichiers (Mapping)

Une autre contrainte industrielle est que nous travaillons sur une copie miroir ("mirror") des scripts du client. Dans cet environnement, les fichiers ne portent pas leur nom d'origine mais sont nommés par des identifiants numériques uniques (ex : `68002.nvn`).
Solution : Nous avons intégré une couche logicielle de résolution qui utilise un fichier index `mapping.txt`. Ce système traduit à la volée les ID en chemins intelligibles pour l'utilisateur (ex : `68002` ↔ `/utilities/preprocess/Forecast Autodiff`), rendant les réponses de l'agent lisibles.

5.2.4 • L'instabilité du raisonnement agentique

L'agent fait preuve d'une certaine variabilité, inhérente à la nature probabiliste des LLM. Sur une même question, il peut parfois trouver la réponse en deux étapes logiques, et d'autres fois errer dans une boucle de réflexion longue, voire halluciner totalement ou échouer à conclure. Nous travaillons actuellement sur l'ajustement de la *température* du modèle (paramètre contrôlant la créativité) pour canaliser ce comportement, mais la stabilisation du graphe de raisonnement reste un défi ouvert.

5.3 Vers une industrialisation : Métriques et Modèles

Au-delà de la faisabilité technique, nous entamons une réflexion sur la viabilité industrielle de notre solution.

5.3.1 • Définition de métriques de performance

Jusqu'à présent, notre évaluation de la performance du workflow était essentiellement empirique : nous jugions "à l'œil" si le temps de réponse était acceptable ou si l'agent semblait "trop réfléchir". Pour passer à un système industrialisable, nous devons définir et suivre des métriques quantitatives claires :

- **Latence de bout en bout :** Mesurer le temps exact entre la requête utilisateur et la réponse finale.
- **Temps de construction de l'index :** Mesurer le temps nécessaire pour parser, chunker et vectoriser l'ensemble du corpus. Une indexation trop lente (plusieurs minutes) rendrait impossible l'intégration dans un cycle de développement rapide (CI/CD).
- **Efficacité des tokens :** Comptabiliser le nombre de tokens générés par rapport à la pertinence de la réponse. Un agent qui "blablate" trop dans sa chaîne de pensée coûte cher inutilement.
- **Taux de succès au premier essai :** Pourcentage de fois où l'agent trouve la réponse sans avoir besoin de se corriger (backtracking).

5.3.2 • Choix des modèles et contraintes de coûts

Un point crucial de notre démarche actuelle est l'utilisation exclusive de modèles accessibles via des API gratuites ou à très bas coût pour le développement. Nous nous appuyons principalement sur l'API de **Mistral AI** ainsi que celle de **Groq**, qui nous permet d'inférer des modèles open-weights performants tels que **Llama 3** ou **Qwen** avec une latence très faible.
Nous avons fait le choix conscient de ne pas dépendre immédiatement de modèles propriétaires coûteux comme GPT-4 (OpenAI) pour la boucle principale de l'agent. Cela nous force à optimiser notre architecture : si notre système fonctionne avec un modèle "moyen" comme Llama, il sera d'autant plus performant avec un modèle

"état de l'art". Cependant, le passage à l'échelle nécessitera de comparer le rapport coût/performance de ces différentes options.

5.4 Pistes d'amélioration avancées pour le RAG

Nous avons identifié une limitation intrinsèque aux modèles d'embedding génériques open-source que nous utilisons, tels que `sentence-transformers/all-MiniLM-L6-v2`. Ces modèles, entraînés sur du langage naturel, peinent parfois à capturer la sémantique structurelle du code (liens logiques, flux de données), entraînant des performances de récupération (Retrieval) sous-optimales.

Pour pallier cela, nous expérimentons actuellement deux pistes prometteuses :

1. **L'indexation de résumés (Summary Embeddings) :** L'idée est de demander à un LLM tiers de générer un résumé textuel explicite pour chaque chunk de code, et d'indexer ce résumé plutôt que le code brut. Cela permettrait à l'embedder de travailler sur du texte qu'il "comprend" mieux. Cependant, cette approche se heurte actuellement à un obstacle majeur : la génération de résumés pour des milliers de chunks via des API gratuites (limitées en débit) rend le temps de construction de l'index prohibitif. Nous étudions des moyens d'optimiser ce processus ou de le réserver aux parties critiques du code.
2. **Le Re-ranking (Reclassement) :** Pour compenser le manque de précision de la recherche initiale, nous envisageons d'implémenter une étape de *Re-ranking*. Au lieu de sélectionner seulement les 5 meilleurs chunks (Top-K=5), nous en récupérerions une centaine (Top-K=100), puis un modèle spécialisé (Cross-Encoder) viendrait les reclasser finement en fonction de leur pertinence contextuelle par rapport à la question. Cela réduirait le risque de manquer un document crucial (faux négatif) lors de la première passe.

5.5 Feuille de route pour la finalisation

Il nous reste trois mois pour transformer ce prototype fonctionnel en un outil robuste et utilisable par les équipes de Lokad. Nos priorités sont clairement définies :

1. **Optimisation du Parser (Pré-résolution) :** Plutôt que de résoudre les chemins dynamiques (*Dynamic Paths*) à chaque requête, nous souhaitons déplacer cette logique en amont, dans la phase de Parsing initiale. Le but est que le *Semantic Chunker* indexe directement les chemins résolus. Cela rendrait l'exécution du GREP quasi-instantanée.
2. **Amélioration de l'Expérience Développeur (DX) :** Le mode "verbose" actuel, indispensable pour le débogage, produit un mur de texte difficile à lire dans le terminal. Nous travaillons sur une interface de terminal améliorée (TUI) utilisant des librairies Python modernes comme `rich`. L'objectif est de rendre le suivi du raisonnement de l'agent (les étapes de réflexion, les outils appelés, les erreurs rencontrées) visuellement clair et agréable pour le développeur.
3. **Stabilisation du Système :** L'objectif principal reste de réduire la variance des réponses. Cela passera par un réglage fin des prompts systèmes (System Prompts) et l'ajout de garde-fous stricts (*max_iterations*) dans LangGraph pour éviter les boucles infinies coûteuses.

Le projet est en bonne voie. L'architecture technique est stable, et les efforts se concentrent désormais sur l'optimisation de la performance (résolution des paths) et la fiabilité des réponses (stabilité de l'agent).

RÉFÉRENCES

[1] LOKAD. *Documentation officielle d'Envision, le DSL de la Supply Chain.* [En ligne]. Disponible sur : `https://docs.lokad.com/` [Consulté le 17/09/2025].

[2] LangChain AI. *LangGraph : Building stateful, multi-actor applications with LLMs.* Documentation technique. [En ligne]. Disponible sur : `https://langchain-ai.github.io/langgraph/`

[3] Promotion X2023 (PSC). *LLM-DSL-Gen : Génération de code DSL par LLM.* Repository GitHub. [En ligne]. Disponible sur : `https://github.com/SaturnTsen/LLM-DSL-Gen`

[4] VASWANI, A., et al. (Google Brain). *Attention Is All You Need.* Advances in Neural Information Processing Systems (NIPS), 2017. (Papier fondateur de l'architecture Transformer).

[5] LEWIS, P., et al. (Facebook AI Research). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS 2020. (Papier fondateur du paradigme RAG).

[6] TAUMAN KALAI, A., et al. *Why Language Models Hallucinate.* 2025. [En ligne]. Disponible sur : `https://arxiv.org/abs/2509.04664`

[7] GAO, Y., et al. *Retrieval-Augmented Generation for Large Language Models : A Survey.* 2023. [En ligne]. Disponible sur : `https://arxiv.org/abs/2312.10997`

[8] WHITE, J., et al. *A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT.* Vanderbilt University, 2023.

[9] ZHENG, L., et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.* NeurIPS 2023 Datasets and Benchmarks Track. (Référence pour la méthodologie d'évaluation).

[10] RICE, I. *How I won the Enterprise RAG Challenge.* [En ligne]. Disponible sur : `https://abdullin.com/ilya/how-to-build-best-rag/`

[11] MÜLDER, A. *Large Language Models for Domain-Specific Language Generation : How to Train Your Dragon.* Medium, 2023. [En ligne]. Disponible sur : `https://medium.com/itemis/large-language-models-for-domain-specific-language-generation-how-to-train-your-dragon`

[12] PRATAP, S., et al. *The fine art of fine-tuning : A structured review of advanced LLM fine-tuning techniques.* ScienceDirect, 2025.